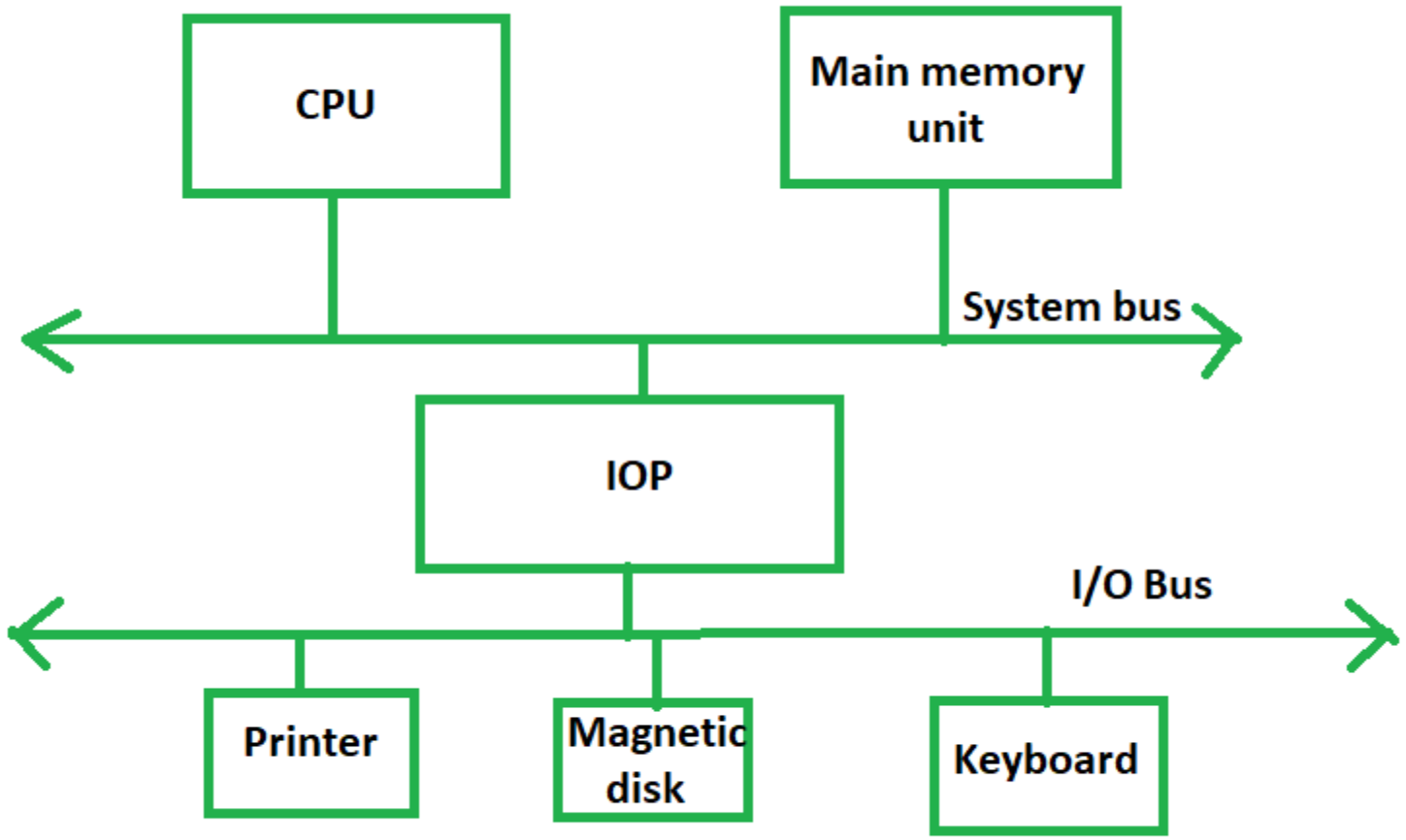
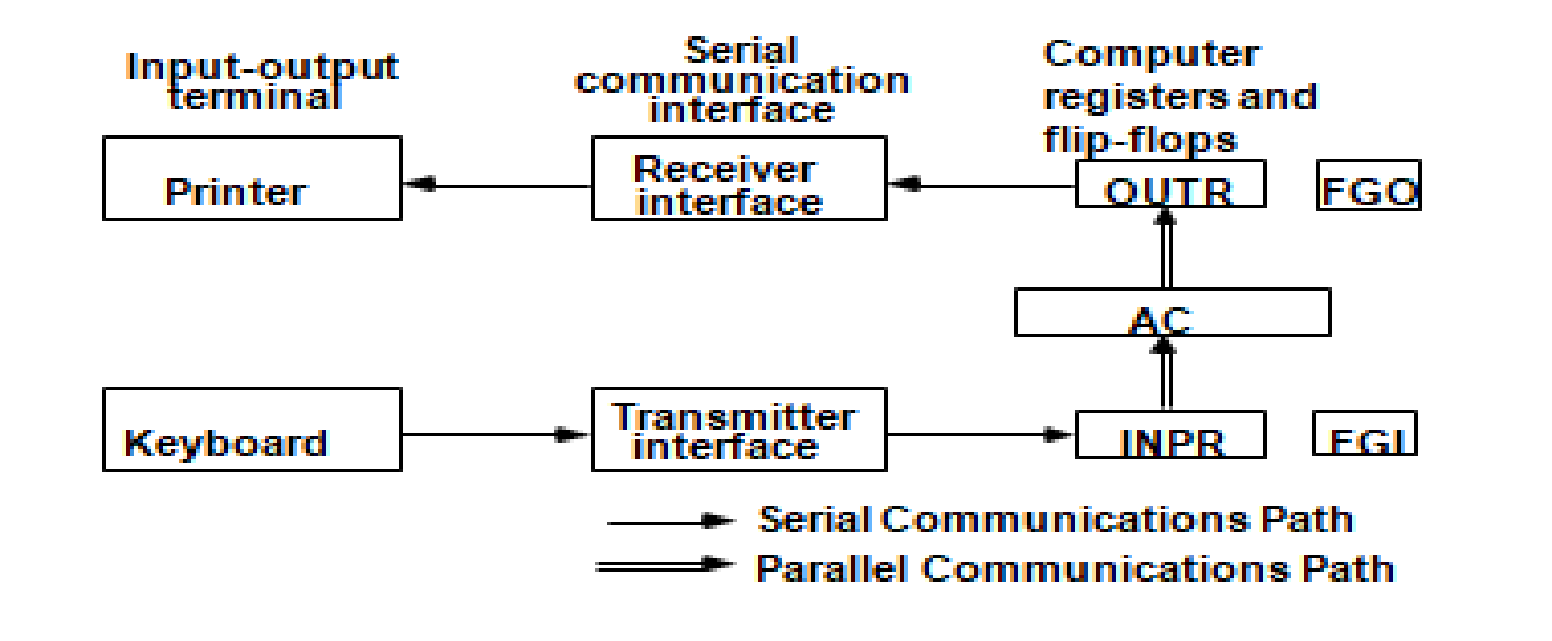
Example



Input-Output Processor

Overview

- Peripheral Devices
- Input-Output Interface
- Asynchronous Data Transfer
- Modes of Transfer
- Priority Interrupt
- Direct Memory Access
- Input-Output Processor
- Serial Communication


Input-output terminal
Serial communication interface
Computer registers and flip-flops
Printer
Receiver interface
OUTR
FGO
AC
Keyboard
Transmitter interface
INPR
FGI
Serial Communications Path
Parallel Communications Path

Input Output Organization

I/O Subsystem

- Efficient Mode Of Communication
  - between central system and outside environment
- Programmed, Input Data, Process, Record, Display Results.
- Slow Keyboard →Processor Idles Waiting For Slow Input.
- time of waiting is more for information to arrive
- E.G. Magnetic Tapes, Disks

Peripheral Devices

- Devices under direct control of computer are
  →connected on-line.
- Peripherals →I/O devices attached to the computer
- There are three types of peripherals :
  - Input peripherals
  - Output peripherals
  - Input-output peripherals

Peripheral Devices

Peripheral (or I/O Device)

Monitor →(*Visual Output Device*) : CRT, LCD

KeyBoard →(*Input Device*) : light pen, mouse, touch screen, joy stick, digitizer

Printer →(Hard Copy Device) : **Daisy wheel**, **dot matrix** and **laser printer**

Storage Device → Magnetic tape, magnetic disk, optical disk

Peripheral Devices

Input Devices

- Keyboard
- Optical input devices
  - Card Reader
  - Paper Tape Reader
  - Bar code reader
  - Optical Mark Reader
- Magnetic Input Devices
  - Magnetic Stripe Reader (ATM)
- Screen Input Devices
  - Touch Screen
  - Light Pen
  - Mouse
-

Output Devices

- Card Puncher, Paper Tape Puncher
- CRT
- Printer (Daisy Wheel, Dot Matrix, Laser)
- Plotter

ASCII Alphanumeric Characters

Input and output devices that communicate with people and the computer are usually involved in the transfer of alphanumeric information to and from the device and the computer. The standard binary code for the alphanumeric characters is ASCII (American Standard Code for Information Interchange). It uses seven bits to code 128 characters as shown in Table 11-1. The seven bits of the code are designated by b_1 through b_7, with b_7 being the most significant bit. The letter A, for example, is represented in ASCII as 1000001 (column 100, row 0001). The ASCII code contains 94 characters that can be printed and 34 nonprinting characters used for various control functions. The printing characters consist of the 26 uppercase letters A through Z, the 26 lowercase letters, the 10 numerals 0 through 9, and 32 special printable characters such as %, *, and $.

TABLE 11-1 American Standard Code for Information Interchange (ASCII)

$b_4 b_3 b_2 b_1$	$b_7 b_6 b_5$ 000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	‘	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	″	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	’	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	|
1101	CR	GS	–	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	—	o	DEL

Control characters

NUL	Null	DLE	Data link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronous idle
BEL	Bell	ETB	End of transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
SP	Space	DEL	Delete

Input Output Organization

➢ **ASCII (*American Standard Code for Information Interchange*)**

- I/O communications usually involves data transfer
- Alphanumeric information from device and the computer.
- Standard binary code for alphanumeric character is **ASCII**
- ASCII Code :
  - It uses 7 bits to code 128 characters (94 printable and 34 non printing)
  - 7 bit - 00 - 7F (*0 - 127*)

Input Output Organization

➢ **ASCII (*American Standard Code for Information Interchange*)**

- ASCII is 7 bits but,
  →most computers manipulate 8 bit quantity as a single unit called byte.

  80 - FF (128 - 255) : Greek, Italic type font

- Three types of control characters: Format effectors, Information separators and communication control

Input Output Organization

- Format effectors control printing layout.
  - Examples: backspace (BS), horizontal tab (HT),
  - Carriage return (CR) is also included.
- Information separators divide data sections, divisions like paragraphs and pages.
  - Examples: record (RS), file separator (FS).
- Communication control: transmission of text between remote terminals STX(Start of text) and ETX(end of text).

Control characters

NUL	Null	DLE	Data link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronous idle
BEL	Bell	ETB	End of transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
SP	Space	DEL	Delete

I/O Interface

- **Provides a method for transferring information**
- **Transfer between internal storage and external I/O devices**
- **(such as memory and CPU registers)**
- **Resolves the *differences* between the computer and peripheral devices**

(1). Peripherals – Electromechanical or Electromagnetic Devices

CPU or Memory - Electronic Device

- **Conversion of signal values required**

I/O Interface

(2). Data Transfer Rate

- Peripherals - Usually slower
- CPU or Memory - Usually faster than peripherals
  - Some kinds of Synchronization mechanism may be needed

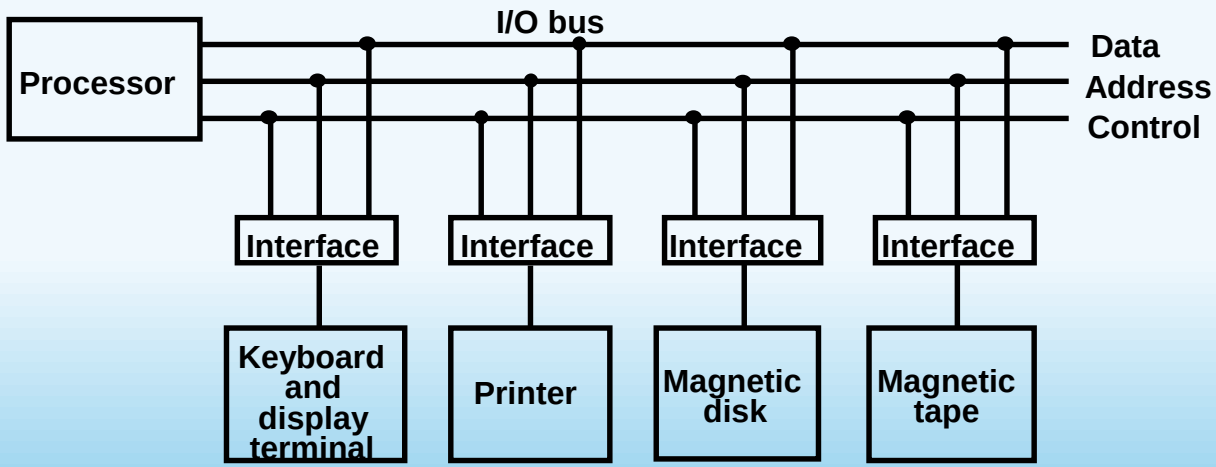
(3). Data formats or Unit of Information

- Peripherals – Byte, Block, …
- CPU or Memory – Word

(4). Operating modes of peripherals may differ

- must be controlled so that not to disturbed other peripherals connected to CPU

I/O Bus and Interface



I/O Bus and Interface

Interface :

- Decodes the device address (device code)

- Decodes the commands (operation)

- Provides signals for the peripheral controller

- Synchronizes the data flow and supervises the transfer rate between peripheral and CPU or Memory

4 types of command interface can receive : control, status, data o/p and data i/p

I/O Bus and Interface

- Control command :
  - issued to activate peripheral and to inform what to do
- Status command :
  - used to test various status condition in interface and peripherals
- data o/p command :
  - causes the interface to respond by transferring data
  - From the bus into one of its registers
- data i/p command :
  - interface receives an item of data from the peripheral
  - Then, places it in its buffer register.

I/O Bus and Memory Bus

Functions of Buses

- ***MEMORY BUS*** **is for information transfers between CPU and the MM**
- ***I/O BUS*** **is for information transfers.**
- **Transfer between CPU and I/O devices through their I/O interface**

I/O Bus and Memory Bus

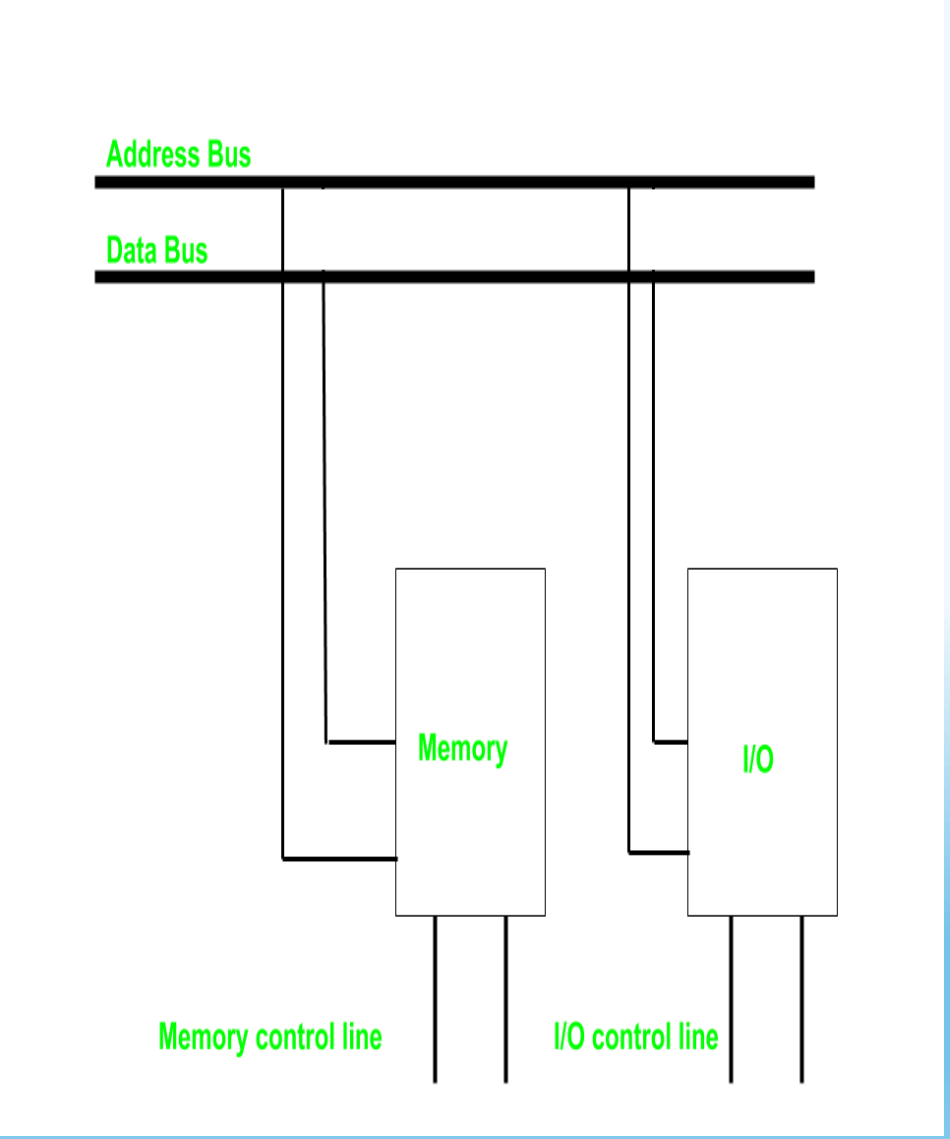
•3 ways to bus can communicate with memory and I/O :

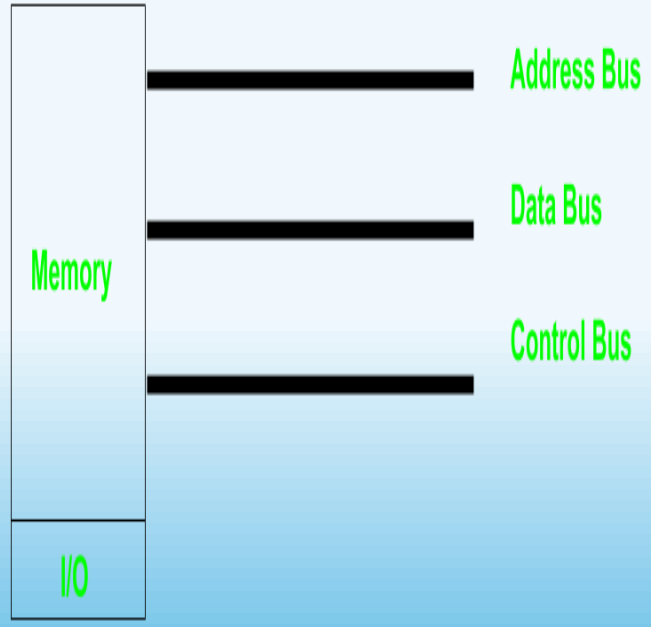
(1). use two separate buses,

- one to communicate with memory , the other with I/O interfaces
  - Computer has independent set of data, address and control bus
  - one for accessing memory and another I/O.
  - done in computers that have separate IOP other than CPU.

(2). Use one common bus for memory and I/O but separate control lines for each

(3). Use one common bus for memory and I/O with common control lines for both


Address Bus
Data Bus
Memory
I/O
Memory control line
I/O control line


Memory
I/O
Address Bus
Data Bus
Control Bus

Isolated vs. Memory Mapped I/O

Isolated I/O

- Many computers use common bus to transfer information between memory or I/O.

- The distinction between memory transfer and I/O transfer is made through separate read and write line.

-In the isolated I/O configuration ,

- the CPU has distinct input and output instructions and each of these instruction is associated with the address of an interface register.

- Distinct input and output instructions
  - each associated with address of interface register

Isolated vs. Memory Mapped I/O

Memory-mapped I/O

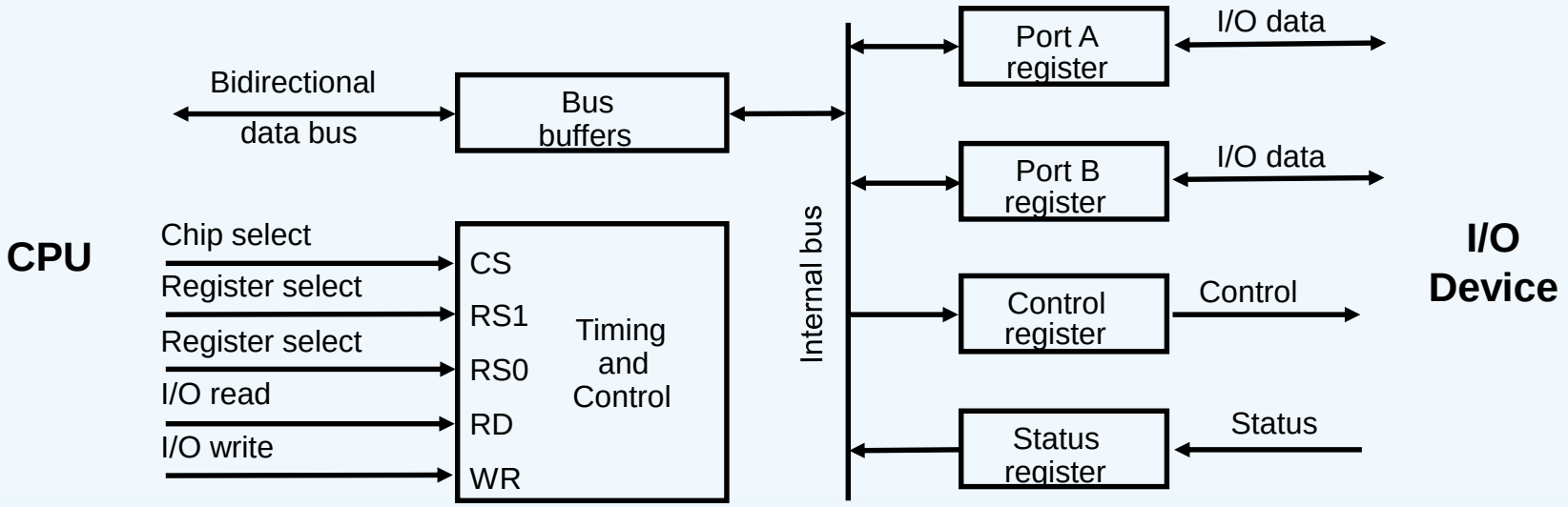
- A single set of read/write control lines
  (no distinction between memory and I/O transfer)

- Memory and I/O addresses share the common address space
  -> reduces memory address range available

- No specific input or output instruction
  -> The same memory reference instructions can be used for I/O transfers

- Considerable flexibility in handling I/O operations

I/O Interface



CS	RS1	RS0	Register selected
0	x	x	None - data bus in high-impedence
1	0	0	Port A register
1	0	1	Port B register
1	1	0	Control register
1	1	1	Status register

Programmable Interface

- **Information in each port can be assigned a meaning depending on the mode of operation of the I/O device→ Port A = Data; Port B = Command;**

- **CPU initializes(loads) each port by transferring a byte to the Control Register**
  → Allows CPU can define the mode of operation of each port
  → *Programmable Port*: By changing the bits in the control register, it is possible to change the interface characteristics

- Peripheral Devices
- Input-Output Interface
- **Asynchronous Data Transfer**
- Modes of Transfer
- Priority Interrupt
- Direct Memory Access
- Input-Output Processor

Asynchronous Data Transfer

→**Synchronous Data Transfer**:

- Uses a common clock for all registers.
- Data transfer occurs simultaneously with clock pulses.
- CPU and I/O Interface operate independently but share a clock for transfer.

→**Asynchronous Data Transfer:**

- Each unit (CPU & Interface) has its own independent clock.
- Control signals are required to synchronize data transfer
- Each unit uses its own private clock for internal registers.
- Asynchronous data transfer between two independent units requires that control signals

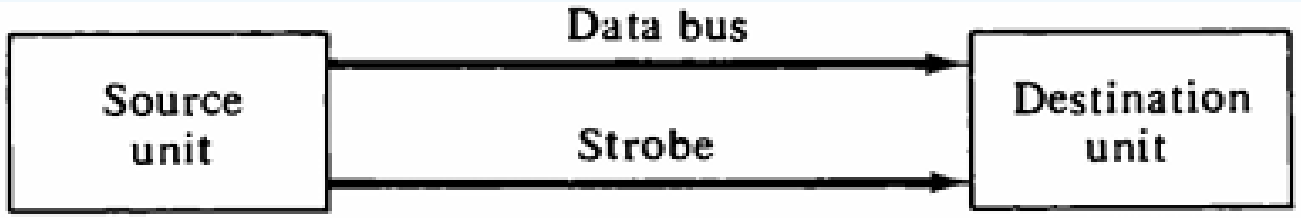
Asynchronous Data Transfer

Two methods:

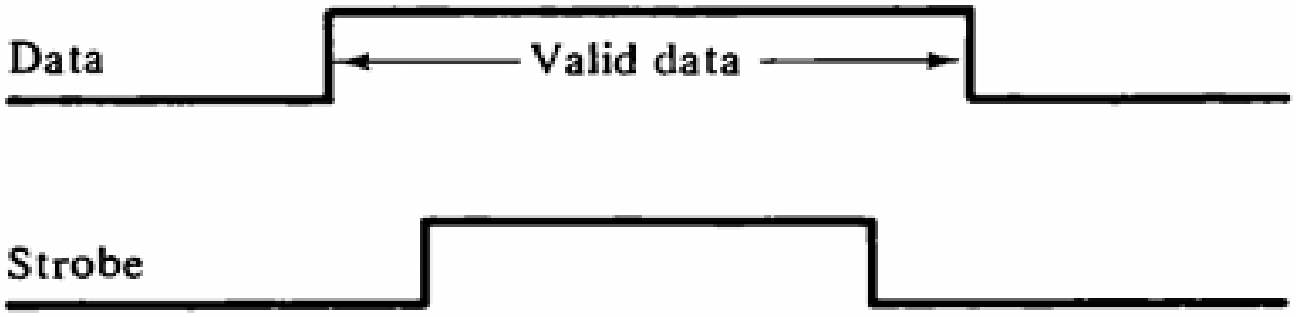
- STROBE: A control signal indicates when data is transmitted.
- HANDSHAKING: A mutual agreement between units for data transfer

The transfer of data between the CPU and an interface unit is similar to the strobe transfer just described. Data transfer between an interface and an I/O device is commonly controlled by a set of handshaking lines.

Source initiated strobe:

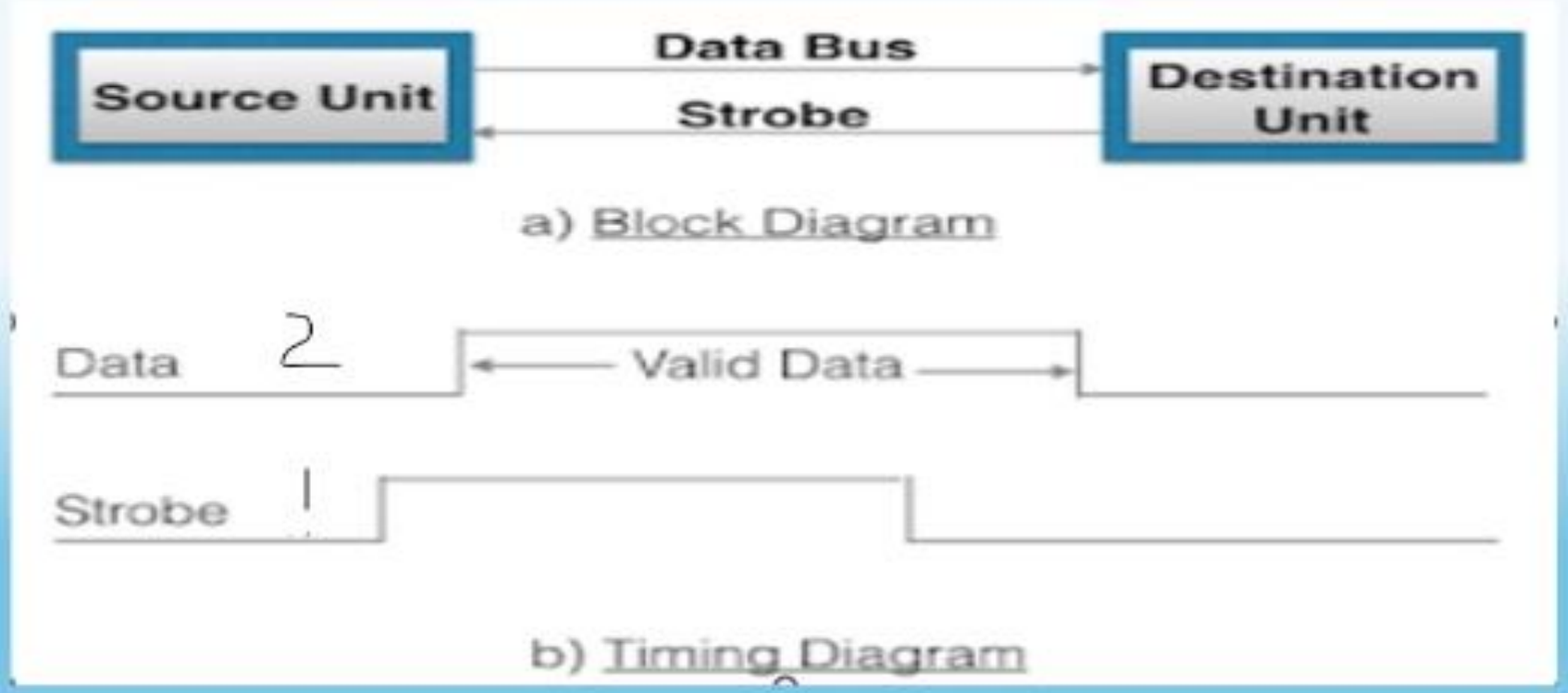


(a) Block diagram



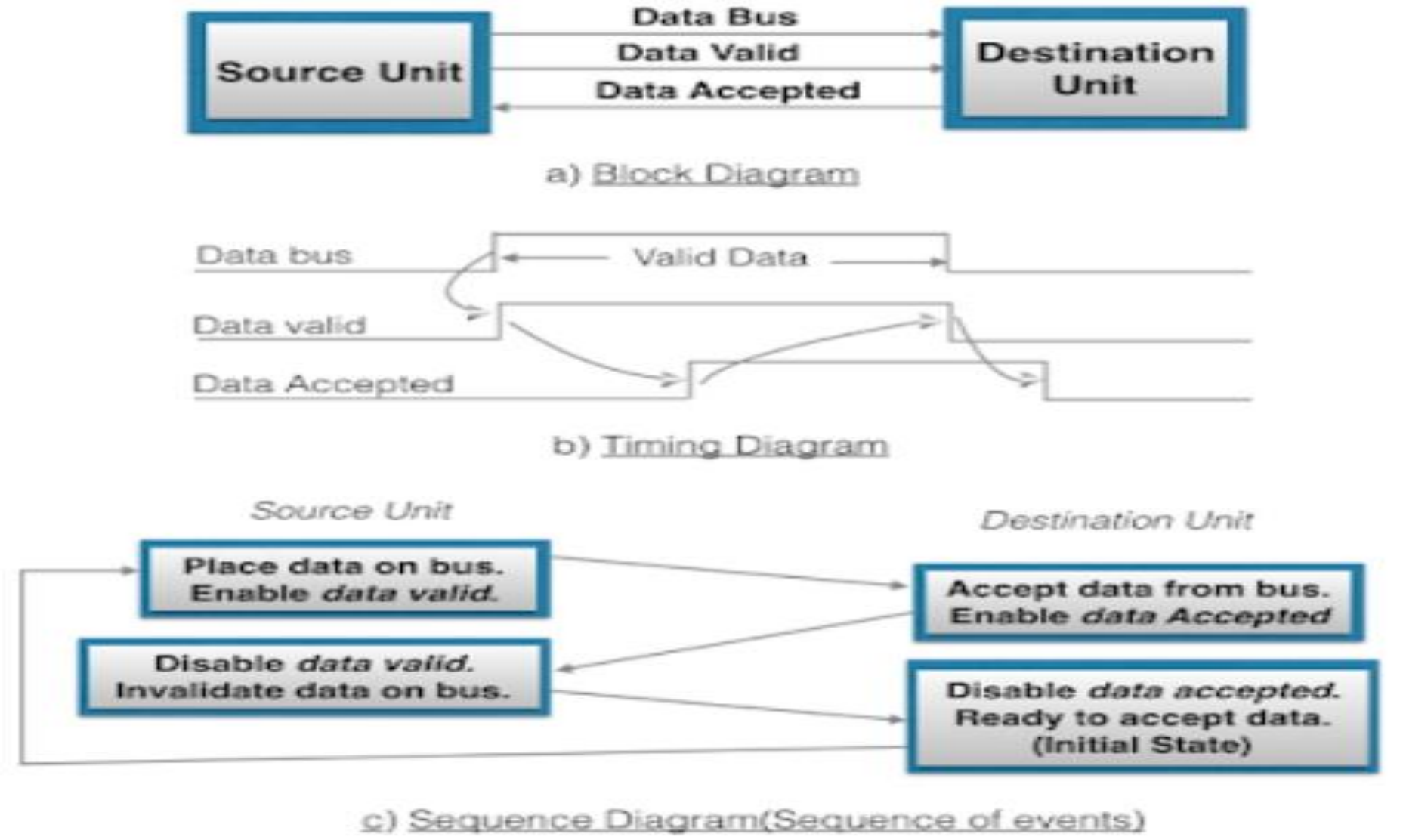
(b) Timing diagram

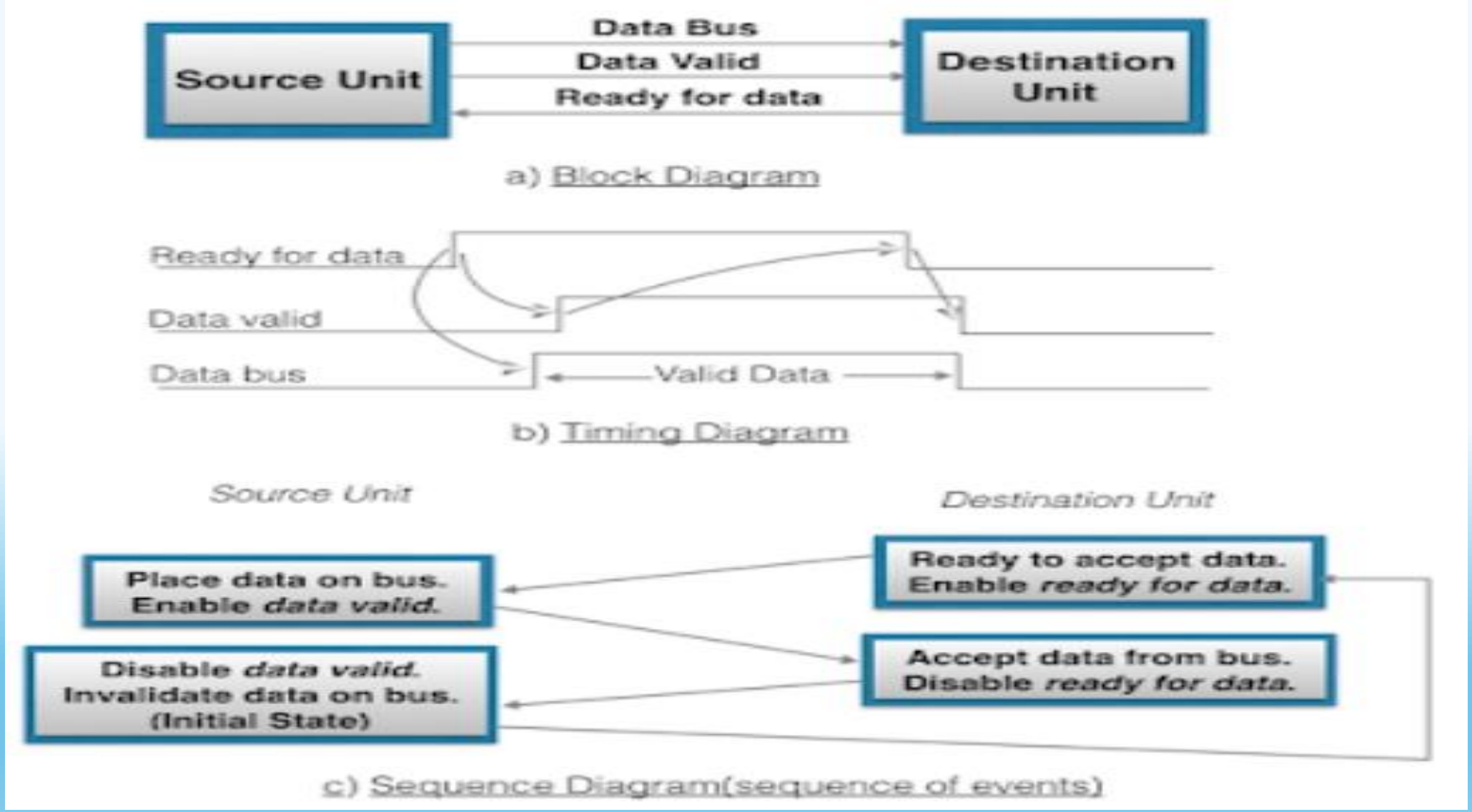
Destination initiated strobe:



The disadvantage of the strobe method is that the source unit that initiates the transfer has no way of knowing whether the destination unit has actually received the data item that was placed in the bus. Similarly, a destination unit that initiates the transfer has no way of knowing whether the source unit has actually placed the data on the bus. The handshake method solves this problem

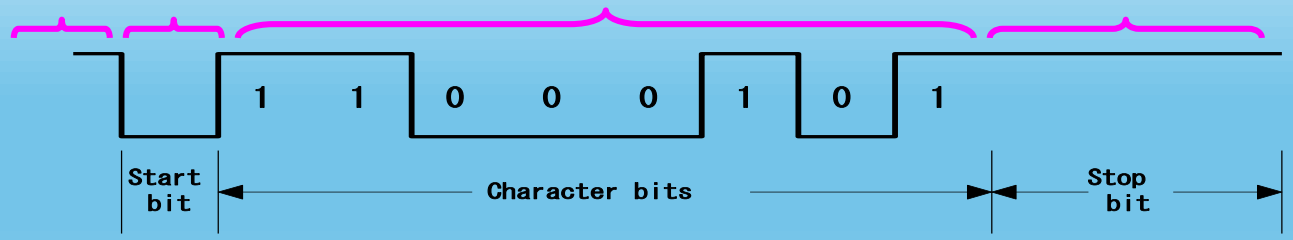
Source initiated handshaking:





Timeout : If the return handshake signal does not respond within a given time period, the unit assumes that an error has occurred.

- Asynchronous Serial Transfer
  - Synchronous transmission :
    - The two unit share a common clock frequency
    - Bits are transmitted continuously at the rate dictated by the clock pulses
  - Asynchronous transmission :
    - Binary information sent only when it is available and line remain idle otherwise
    - Special bits are inserted at both ends of the character code
    - Each character consists of three parts :
      - 1) start bit : always “0”, indicate the beginning of a character
      - 2) character bits : data
      - 3) stop bit : always “1”

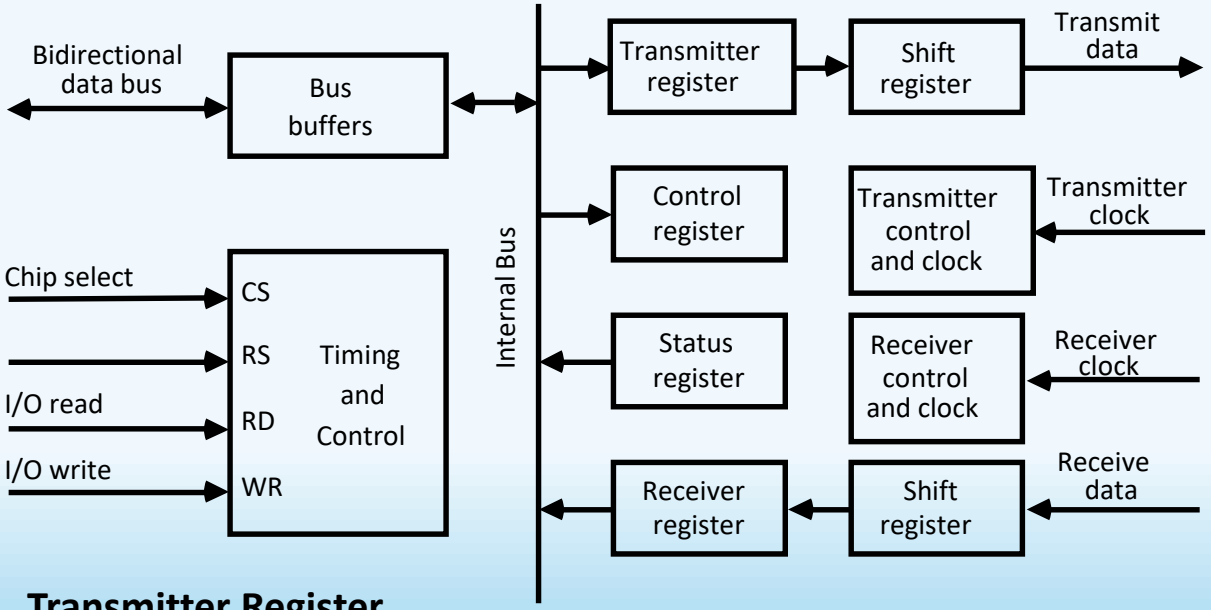


- Asynchronous transmission rules :
  - ① When a character is not being sent, the line is kept in the 1-state
  - ② The initiation of a character transmission is detected from the start bit, which is always “0”
  - ③ The character bits always follow the start bit
  - ④ After the last bit of the character is transmitted, a stop bit is detected when the line returns to the 1-state for at least one bit time

- Baud Rate : Data transfer rate in bits per second
  - 10 character per second with 11 bit format = 110 bit per second

Universal Asynchronous Receiver Transmitter

A typical asynchronous communication interface available as an IC



CS	RS	Oper.	Register selected
0	x	x	None
1	0	WR	Transmitter register
1	1	WR	Control register
1	0	RD	Receiver register
1	1	RD	Status register

Transmitter Register

- Accepts a data byte(from CPU) through the data bus
- Transferred to a shift register for serial transmission

Receiver Register

- Receives serial information into another shift register
- Complete data byte is sent to the receiver register

Status Register Bits

- Used for I/O flags and for recording errors

Control Register Bits

- Define baud rate, no. of bits in each character, whether to generate and check parity, and no. of stop bits

- UART stands for?

a. Universal Asynchronous Sender Receiver
b. Universal Asynchronous Receiver Transmitter
c. Universal Synchronous Receiver Transmitter
d. None Of These.

Integrated circuits are available which are specifically designed to provide the interface between the computer and similar interactive terminals. Such a circuit is called Universal Asynchronous Receiver Transmitter(UART)

Overview

- Peripheral Devices
- Input-Output Interface
- Asynchronous Data Transfer
- **Modes of Transfer**
- **Priority Interrupt**
- Direct Memory Access
- Input-Output Processor

Modes of Transfer

- Binary information received from external device is usually stored in memory.

- Information transferred from central computer into an external device originates in the memory unit.

- The CPU merely execute I/O instructions and may accept data temporarily but ultimate source or destination is the Memory Unit.

Modes of Transfer

- Data transfer between central computer and I/O devices may be handled in a variety of modes.

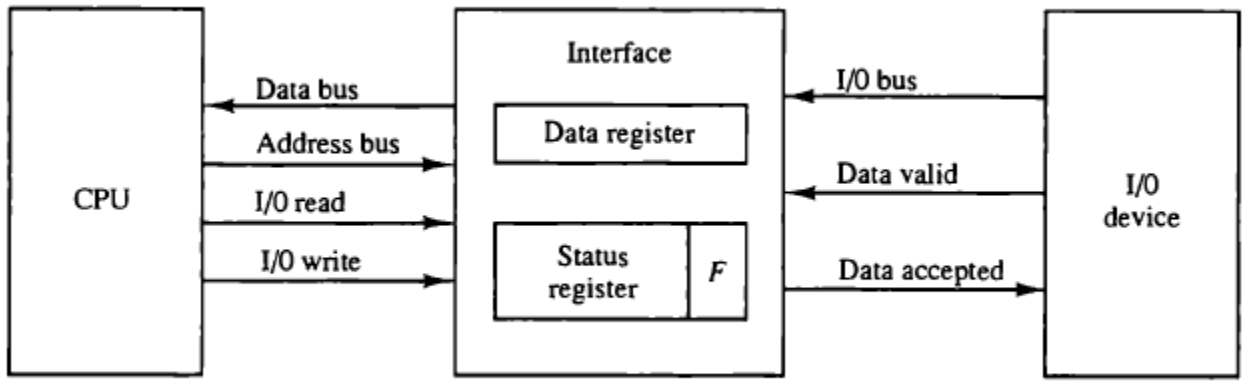
- Some modes use CPU as intermediate path,

  - others transfer data directly to and from memory unit.

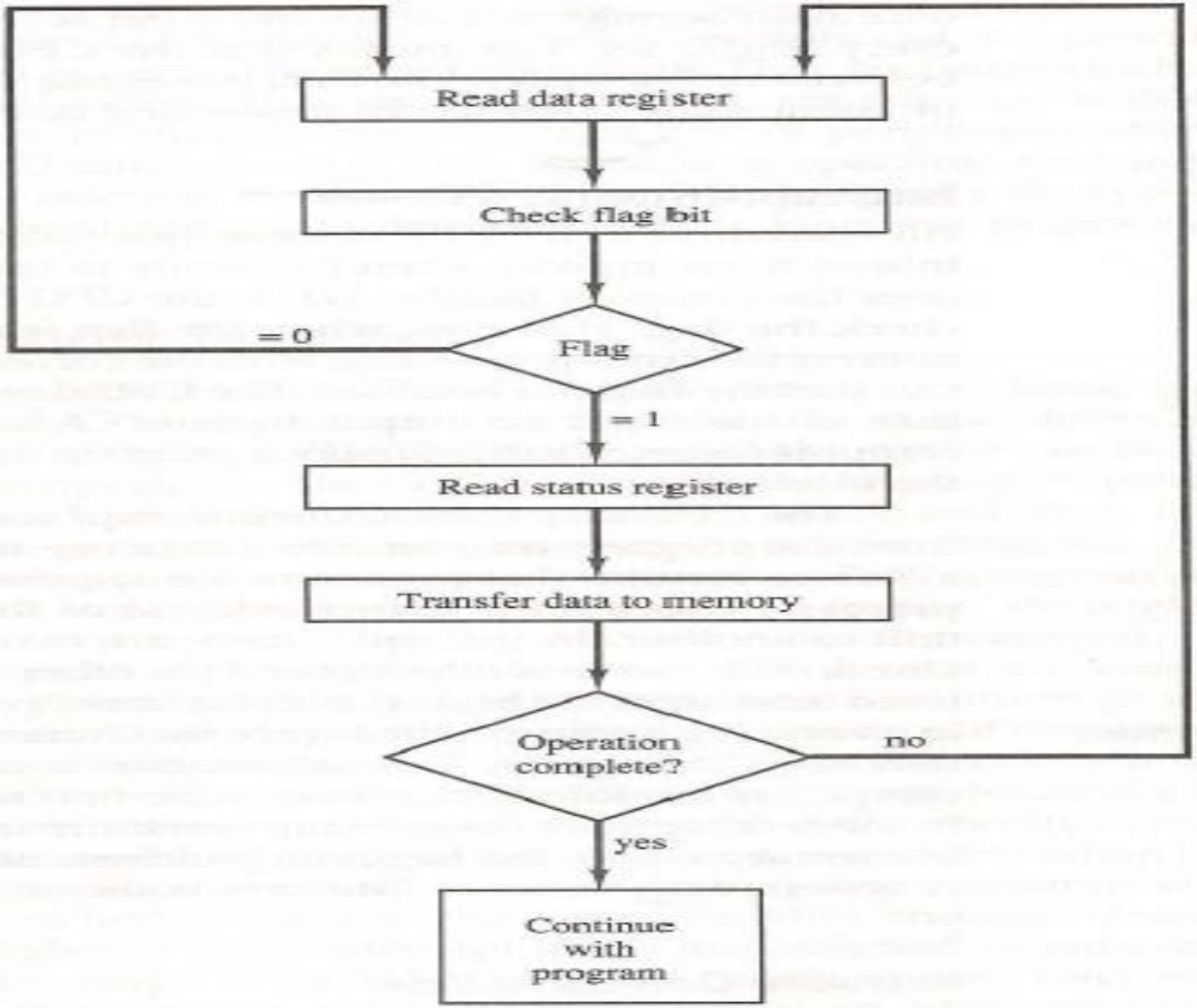
- Data Transfer to or from peripheral can be handled in one of three possible modes :

  - **Programmed I/O**
  - **Interrupt-Initiated I/O**
  - **Direct Memory Access (DMA)**

Modes of Transfer – Programmed I/O



F = Flag bit


Read data register
Check flag bit
Flag
= 0
= 1
Read status register
Transfer data to memory
Operation complete?
no
yes
Continue with program

Programmed I/O

- Programmed I/O operations are the result of I/O Instructions written in computer program.

- Each data item transfer is initiated by an instruction in the program.

- Usually, transfer is to and from a CPU register to peripheral.

- Other instructions are needed to transfer data to and from CPU and Memory

- Transferring data under program control requires constant monitoring of the peripheral by CPU.

- In programmed I/O method,
  - CPU stays in a program loop until the I/O unit indicated that it is ready for data transfer.
  - This is a time consuming process since it keeps the processor busy needlessly.
  - It can be avoided by using **Interrupt** facility and special commands
  - **Interrupt** to inform the interface to issue an interrupt request signal when data are available for the device.

Interrupted I/O

- In the programmed I/O method, the CPU stays in a program loop until the I/O unit indicates that it is ready for data transfer.
- This is a time-consuming process since it keeps the processor busy needlessly.
- It can be avoided by using an interrupt facility and special commands to inform the interface to issue an interrupt request signal when the data are available from the device.
- In the meantime the CPU can proceed to execute another program. The interface meanwhile keeps monitoring the device.

Interrupted I/O

- When the interface determines that the device is ready for data transfer, it generates an interrupt request to the computer.
- Upon detecting the external interrupt signal, the CPU momentarily stops the task it is processing,
- Then, Branches to a service program to process the I/O transfer, and then returns to the task it was originally performing.

DMA

1.Programmed I/O: In this method, data moves between the CPU and external devices (like a keyboard, printer, or hard drive). The CPU controls the entire process.

2.Direct Memory Access (DMA): Instead of the CPU handling data transfer, a special hardware unit (DMA controller) transfers data directly between memory and external devices.

3.CPU's Role in DMA: The CPU starts the transfer by giving the DMA controller a memory address and the amount of data to transfer. Then, the CPU moves on to other tasks.

4.How DMA Works: When the DMA is ready to transfer data, it requests permission to use the memory bus (the pathway between memory and other parts of the system).

DMA

5.Memory Controller's Role: Once the memory controller allows access, the DMA controller moves data directly between the external device and memory.

6.Effect on CPU: The CPU temporarily pauses its memory operations while DMA transfers data but resumes immediately after.

7.Why This Matters: Since most external devices (like hard drives) work much slower than the CPU, these data transfers happen less frequently compared to how often the CPU accesses memory.

This approach allows the CPU to work on other tasks while data transfer happens in the background, improving overall system efficiency.

Priority Interrupts

Priority

- Determines which interrupt is to be served first,
  - when two or more requests are made simultaneously

- Also determines which interrupts are permitted to interrupt the computer,
  - while another is being serviced

- Higher priority interrupts can make requests while servicing a lower priority interrupt

Priority Interrupts

A priority interrupt is a system that establishes priority over the various sources to determine

- which condition is to serviced first when two or more requests arrive simultaneously

-which conditions are permitted to interrupt the computer while another request is being serviced

Priority Interrupts

Priority Interrupt by Software **(Polling)**

Polling procedure is used to identify highest priority source by software means

- common branch address for all the interrupts

- **Priority is established by the order of polling the devices(interrupt sources)**

  - highest priority device is tested first and if interrupt is on ,

- control branches to service routine for this source otherwise next lower priority source is tested

Priority Interrupts

- Flexible since it is established by software

- Low cost since it needs a very little hardware

- Very slow

- if there are many interrupt time required to poll may exceed time available to service IO device

Priority Interrupts

Priority Interrupt by Hardware

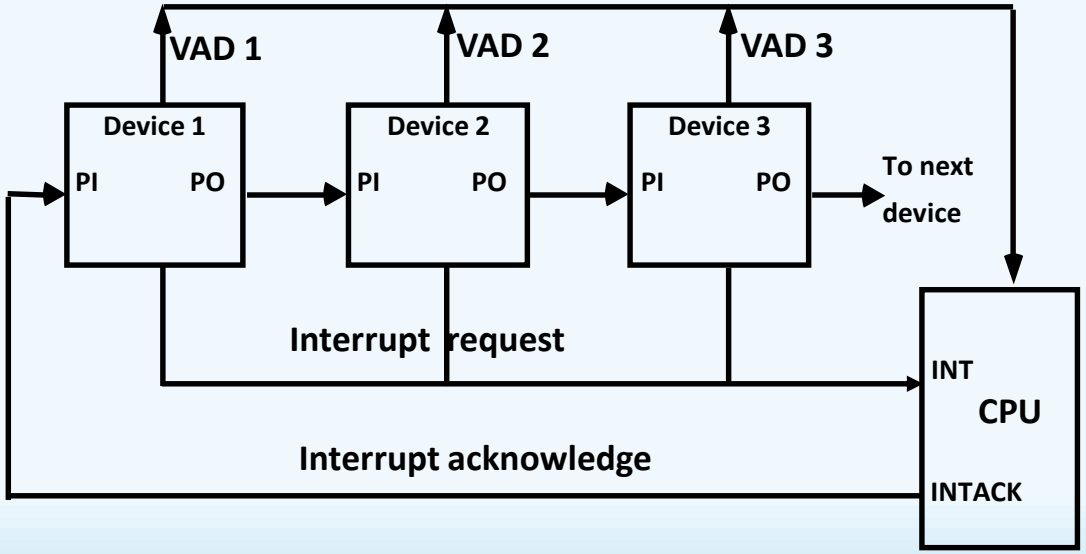
- Require a **priority interrupt manager** which accepts all the interrupt requests to determine the highest priority request

- Fast since identification of the highest priority interrupt request is identified by the hardware

- Fast since each interrupt source has its own interrupt vector to access directly to its own service routine

- Can be addressed using serial or parallel connection of interrupt lines. Example of serial is Daisy chaining Priority

Hardware Priority Interrupts – Daisy Chain



* Serial hardware priority function
* Interrupt Request Line
  - Single common line
* Interrupt Acknowledge Line
  - Daisy-Chain

-Serial connection of all device that request an interrupt

-Device with highest priority placed in first position followed by devices with lower priority and so on.

-Interrupt generated by any device → signals low state interrupt line

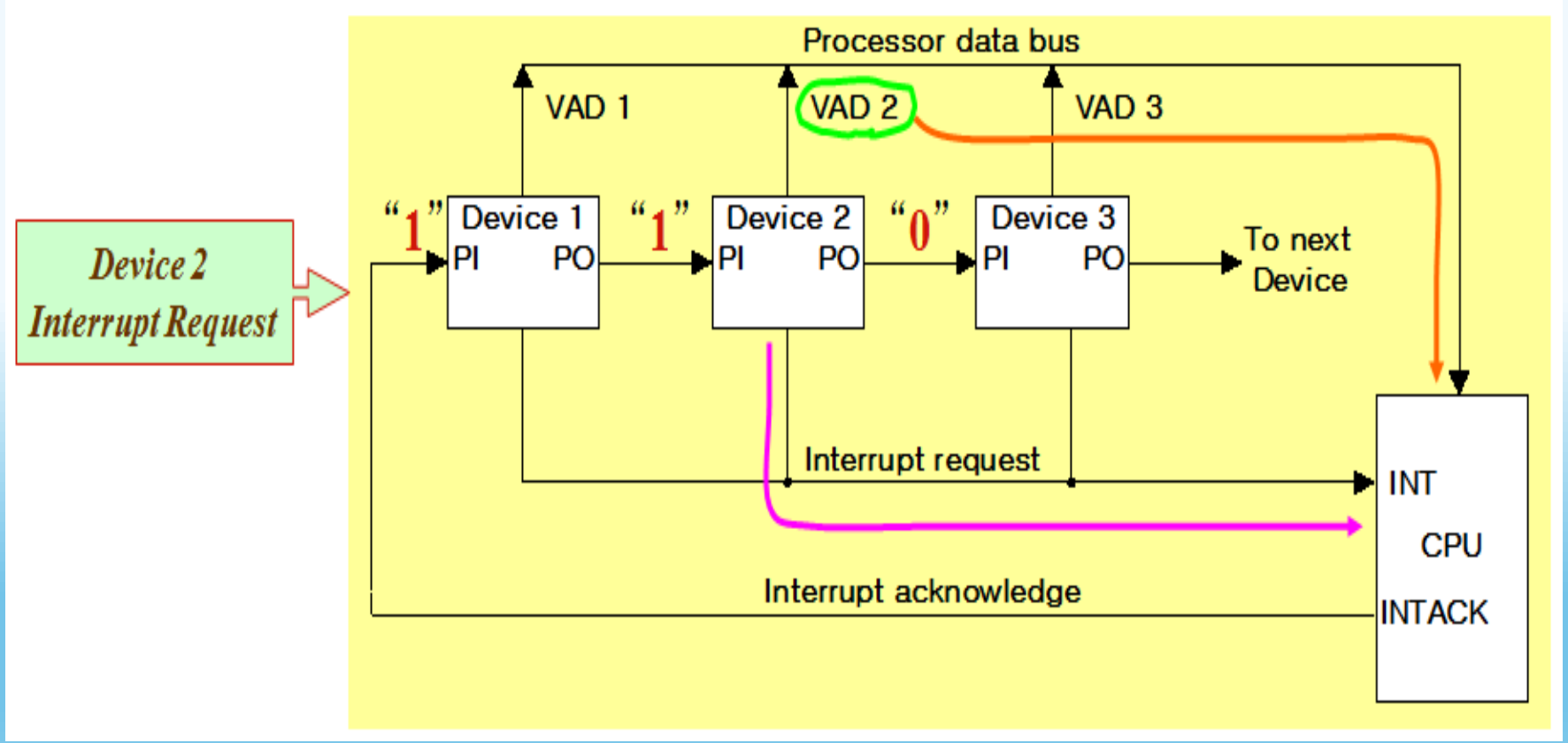
-CPU responds by enabling interrupt acknowledgement (INTACK) line.

- device receives PI=1 and passes to next only when not requesting else PI=0

-Thus device with PI=1 and PO=0 is one with highest priority requesting interrupt

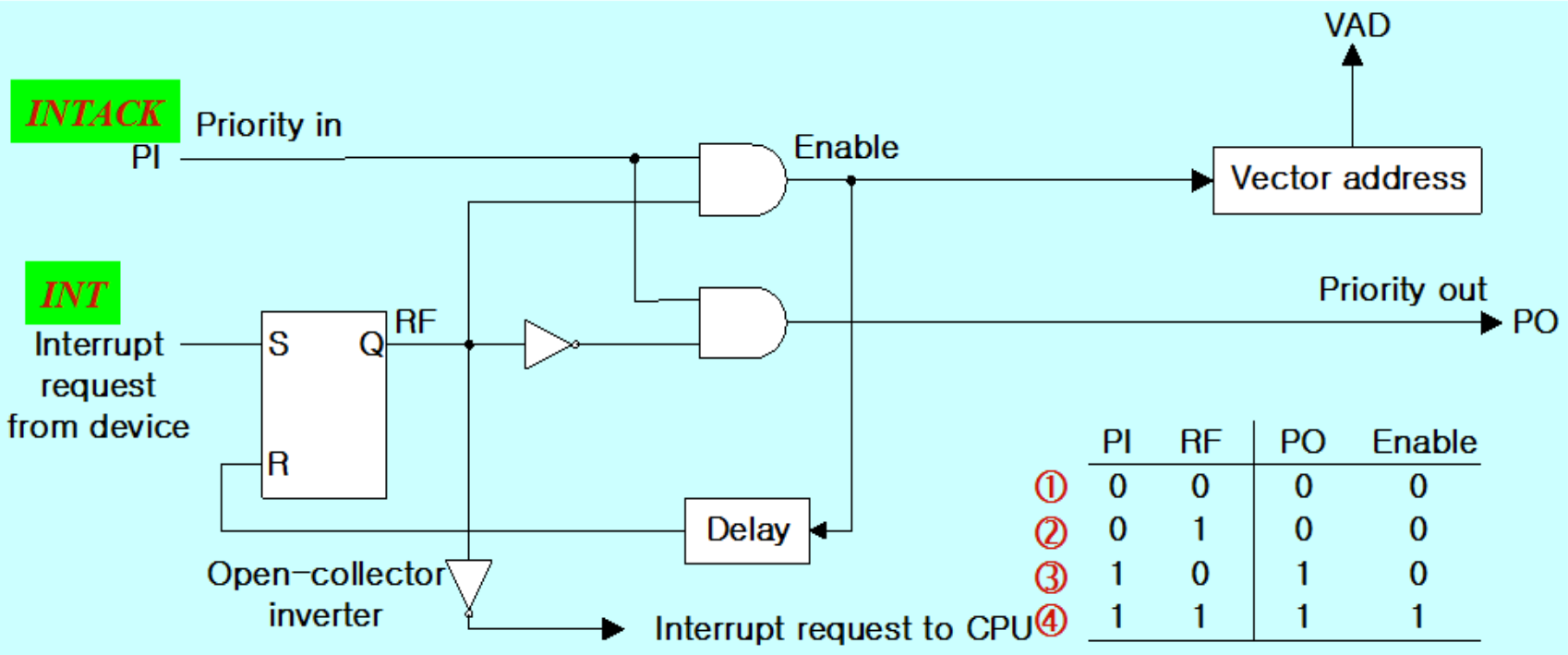
Hardware Priority Interrupts – Daisy Chain

Example: Daisy chain working



Hardware Priority Interrupts – Daisy Chain

- *Fig :*One stage of the daisy-chain priority arrangement



	PI	RF	PO	Enable
①	0	0	0	0
②	0	1	0	0
③	1	0	1	0
④	1	1	1	1

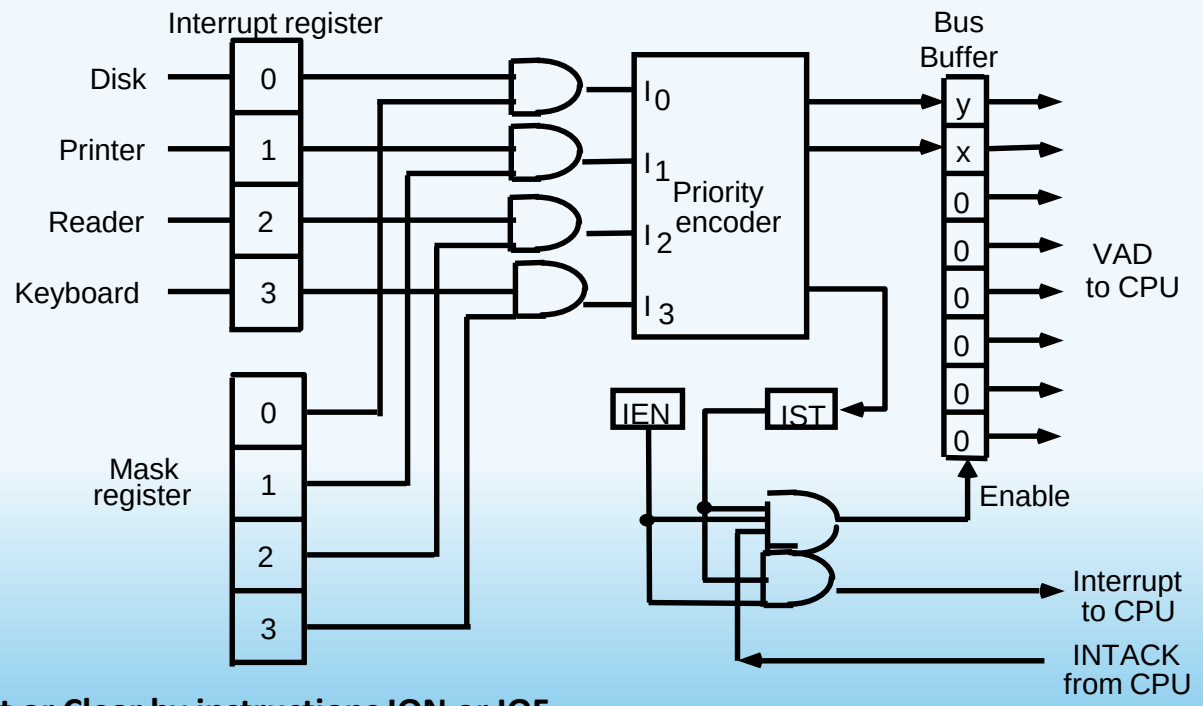
① No interrupt request

② Invalid : interrupt request, but no acknowledge

③ No interrupt request : Pass to other device (*other device requested interrupt*)

④ Interrupt request

Parallel Priority Interrupts



IEN: Set or Clear by instructions ION or IOF
IST: Represents an unmasked interrupt has occurred. INTACK enables tristate Bus Buffer to load VAD generated by the Priority Logic

Interrupt Register:
- **Each bit is associated with an Interrupt Request from different Interrupt Source - different priority level**
- **Each bit can be cleared by a program instruction**

Mask Register:
- **Mask Register is associated with Interrupt Register**
- **Each bit can be set or cleared by an Instruction**

Priority Encoder

Determines the highest priority interrupt when more than one interrupts take place

Priority Encoder Truth table

Inputs				Outputs			
I_0	I_1	I_2	I_3	x	y	IST	Boolean functions
1	d	d	d	0	0	1	
0	1	d	d	0	1	1	
0	0	1	d	1	0	1	$x = I_0' \; I_1'$
0	0	0	1	1	1	1	$y = I_0' \, I_1 + I_0' \, I_2'$
0	0	0	0	d	d	0	$(IST) = I_0 + I_1 + I_2 + I_3$

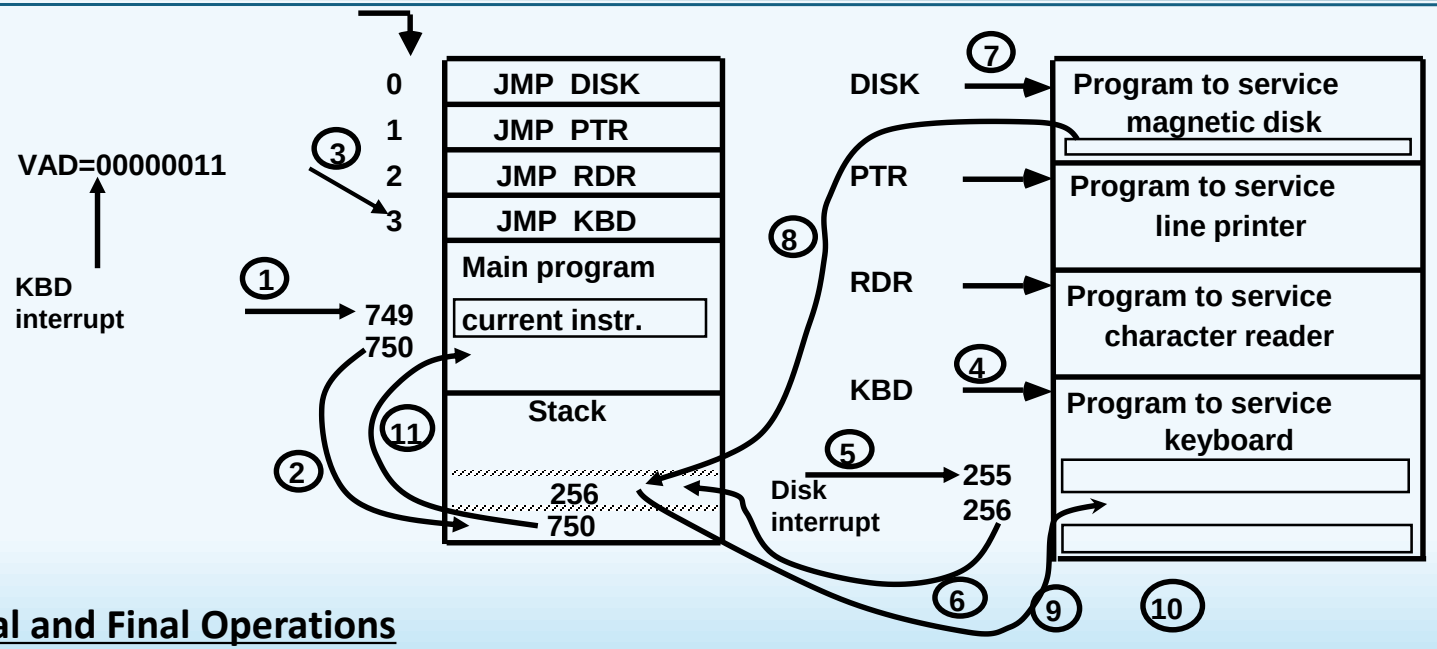
Interrupt Cycle

At the end of each Instruction cycle

- CPU checks IEN and IST
- If IEN • IST = 1, CPU -> Interrupt Cycle

SP ← SP - 1	Decrement stack pointer
M[SP] ← PC	Push PC into stack
INTACK ← 1	Enable interrupt acknowledge
PC ← VAD	Transfer vector address to PC
IEN ← 0	Disable further interrupts
Go To Fetch	to execute the first instruction in the interrupt service routine

Initial and Final Operations



Initial and Final Operations

Each interrupt service routine must have an initial and final set of operations for controlling the registers in the hardware interrupt system

Initial Sequence

[1] Clear lower level Mask reg. bits
[2] IST <- 0
[3] Save contents of CPU registers
[4] IEN <- 1
[5] Go to Interrupt Service Routine

Final Sequence

[1] IEN <- 0
[2] Restore CPU registers
[3] Clear the bit in the Interrupt Reg
[4] Set lower level Mask reg. bits
[5] Restore return address, IEN <- 1

In Daisy Chaining Priority if the device does not have any pending interrupt requests then the value of PI and PO will be

a. PI=0, PO=0
b. PI=0, PO=1
c. PI=1, PO=0
d. PI=1, PO=1

Overview

- Peripheral Devices
- Input-Output Interface
- Asynchronous Data Transfer
- Modes of Transfer
- Priority Interrupt
- **Direct Memory Access**
- **Input-Output Processor**

Direct Memory Access

- Block of data transfer between high speed devices like Disk and Memory

* DMA controller - Interface which takes over the buses to manage the transfer directly between
    - Memory and I/O Device, freeing CPU for other tasks

* CPU initializes DMA Controller by sending memory address and the block size (number of words)

Direct Memory Access

Fig 1: CPU bus signals for DMA transfer (Bus control by DMA)

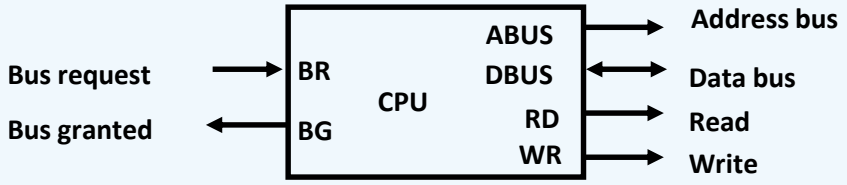
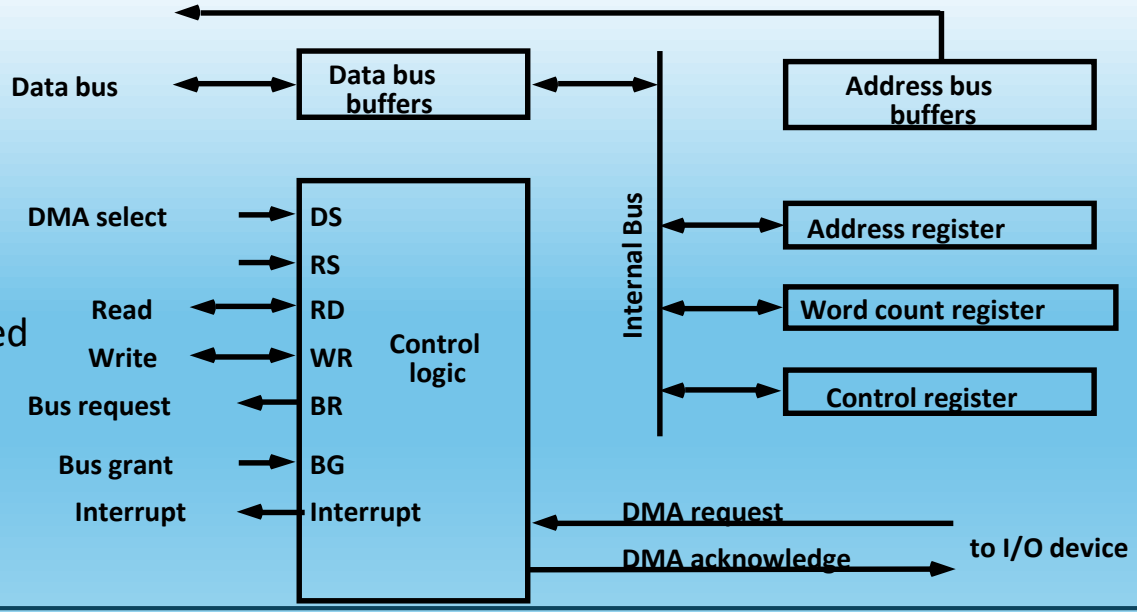


Fig 2: Block diagram of DMA controller



Address register:

Contains an address to specify Desired location in memory

Word count register

Holds no. of words to be transferred

Control register

Specifies the mode of transfer

Direct Memory Access

RD and WR is bidirectional

When BG=0 CPU can communicate with DMA Register (RW info.)
When BG=1 CPU left the buses and DMA can communicate directly with memory

DMA Transfer can be made in several ways

(1) Burst Transfer :

- **a block sequence consisting of memory words is transferred in continuous burst while the DMA controller is master of memory bus**
- **This mode of transfer is needed for fast devices such as magnetic disk where data transmission cannot be stopped or slowed down until an entire block is transferred**

Direct Memory Access

(2) Cycle stealing :

- **Alternative technique called cycle stealing allows DMA controller to transfer one data word at time after which it must return control of the buses to the CPU.**
- **CPU merely delays its operation for one memory cycle to allow the direct memory I/O transfer to "steal" one memory cycle**

DMA I/O Operation

DMA is first initialized by CPU. After that DMA starts and continues to transfer data between memory and peripheral unit until an entire block is transferred.

CPU initializes the DMA by sending following information through data bus:

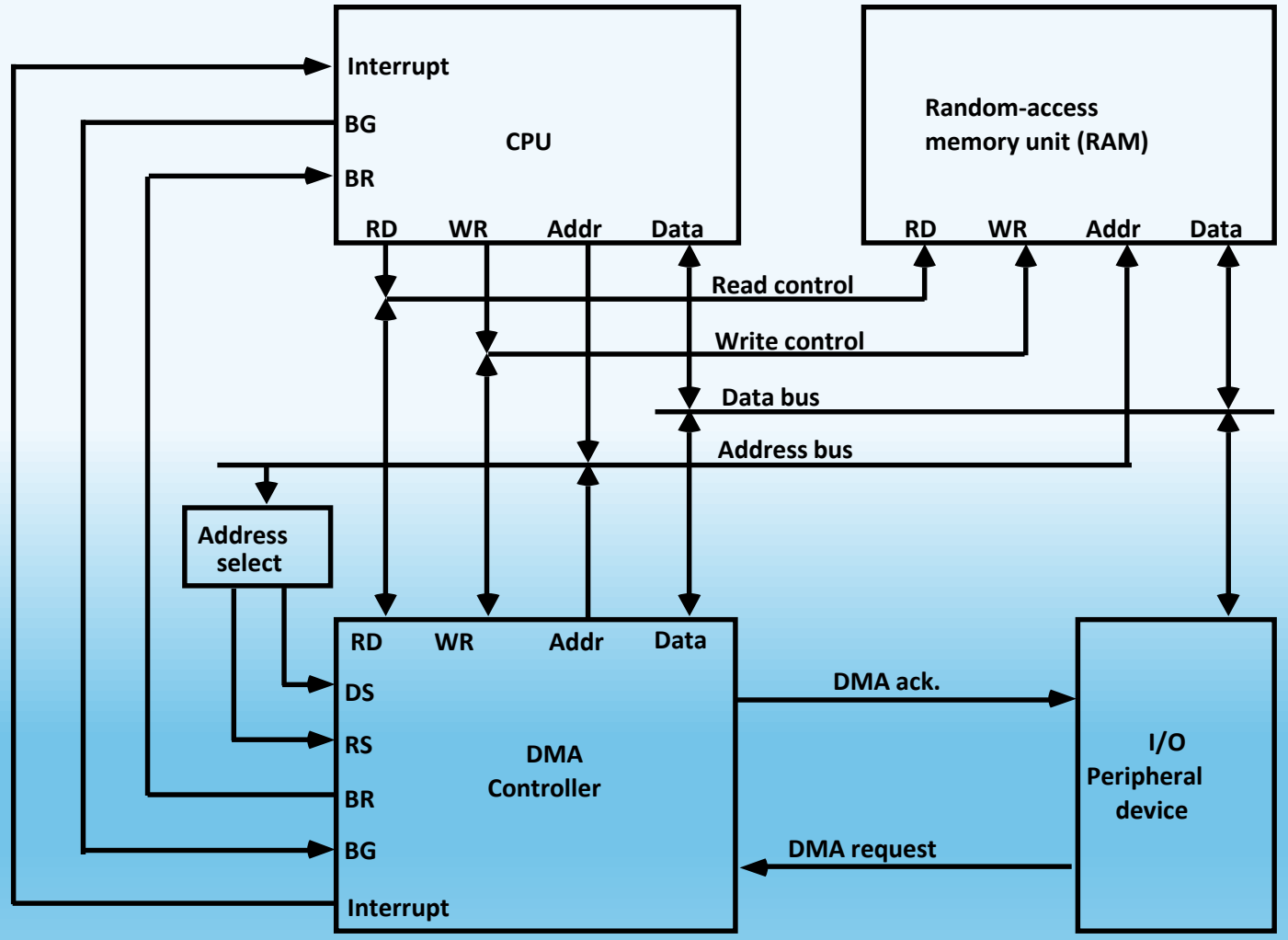
(1) Starting address of the memory block (for read/write)

(2) Word Count (no. of words in memory block)

(3) Control to specify mode of transfer (E.g. read/write)

(4) A control to start DMA Transfer

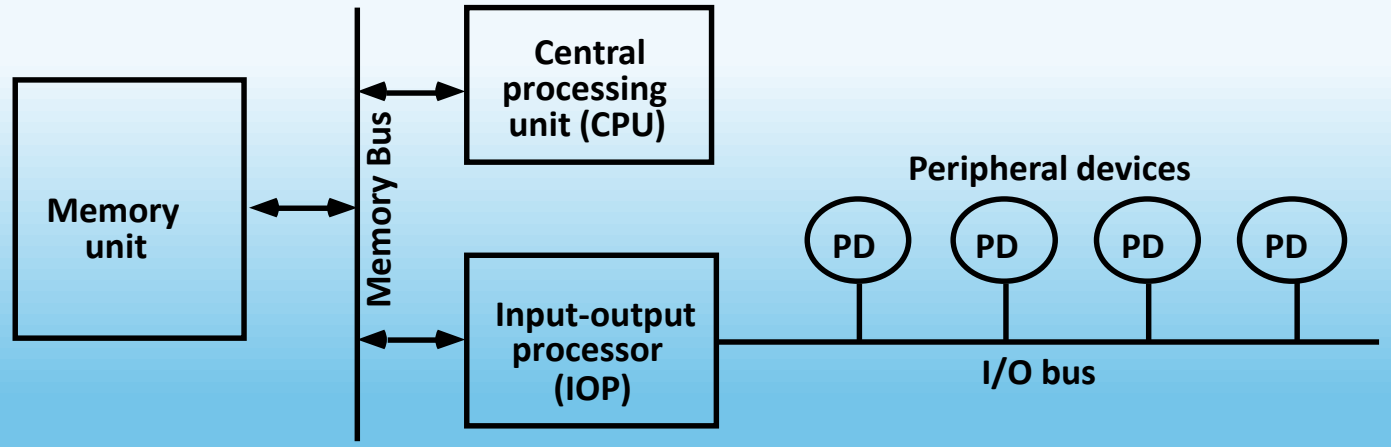
DMA Transfer



I/O Processor - Channel

Channel

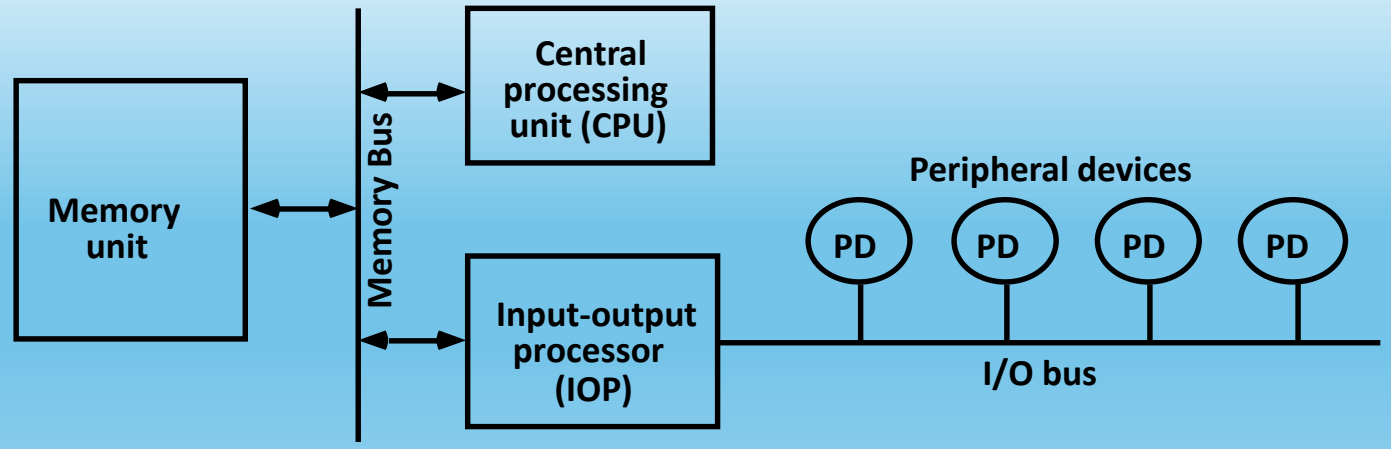
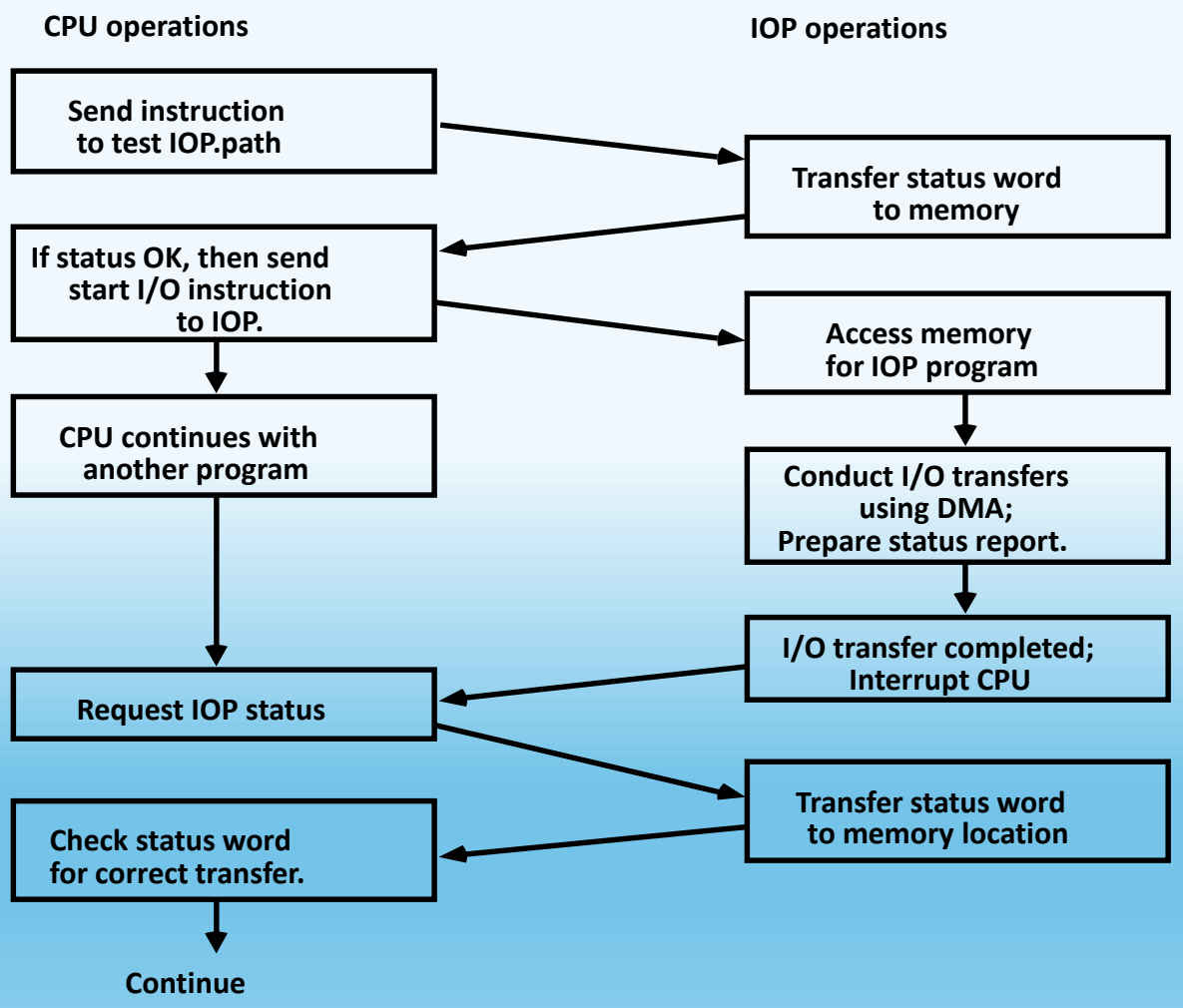
- Processor with direct memory access capability that communicates with I/O devices
- Channel accesses memory by cycle stealing
- Unlike DMA Controller, IOP can fetch and execute its own instruction
- IOP Instructions (Commands) specially designed to facilitate I/O transfer.



I/O Processor - Channel

- Data gathered in IOP at device rate and bit capacity while CPU executing own program
- Transfer between IOP and Device similar to Programmed I/O and transfer between IOP and Memory similar to DMA
- CPU is master while IOP is slave processor

- CPU initiates the channel by executing a channel I/O class instruction and once initiated, channel operates independent of the CPU



Channel CPU Communication